

Support Vector Machine: SVC, Hyperplane, Kernel, dan Parameter Tuning

Materi Week 11 – Supervised Learning

Modul Week 11: Supervised Learning
Topik Support Vector Machine dan SVC
Penulis Danish Rafie Ekaputra
Mata Kuliah Praktikum Data Mining

Tujuan Pembelajaran

Setelah mempelajari materi ini, mahasiswa diharapkan mampu:

1. Menjelaskan basic concept Support Vector Machine untuk supervised learning.
2. Menjelaskan hyperplane, margin, dan support vectors.
3. Membedakan hard margin dan soft margin SVM.
4. Menjelaskan intuisi convex optimization pada SVM.
5. Menggunakan konsep dasar Support Vector Classification (SVC).
6. Menjelaskan kernel trick dan beberapa kernel function umum.
7. Menjelaskan parameter penting pada SVC seperti C , kernel, gamma, degree, dan $class_weight$.
8. Menentukan parameter SVM dengan Grid Search, Random Search, dan Optuna.

Big idea. SVM tidak hanya mencari boundary yang memisahkan class. SVM mencari boundary yang punya *margin* paling aman terhadap data terdekat, lalu mengontrol trade-off antara margin besar dan training error melalui parameter C .

1 Positioning: Kenapa SVM?

Dalam classification, banyak model bisa membuat decision boundary. Pertanyaan utama SVM adalah: dari semua boundary yang mungkin, boundary mana yang paling aman untuk generalization? Jawabannya adalah boundary dengan margin terbesar. Margin besar berarti garis pemisah tidak terlalu dekat dengan data training, sehingga lebih stabil ketika bertemu data baru.

Kondisi	Kenapa SVM Cocok	Catatan
High-dimensional features	SVM efektif saat jumlah fitur besar.	Tetap perlu regularization.
Dataset kecil-menengah	Kernel SVM bisa menangkap boundary non-linear.	Untuk dataset sangat besar, pertimbangkan LinearSVC.
Boundary kompleks	Kernel trick membuat boundary non-linear di input space.	Kernel dan parameter harus dipilih hati-hati.
Butuh margin-based classifier	SVM eksplisit maximize margin.	Sensitif terhadap scaling fitur.

Table 1: Kondisi yang membuat SVM menjadi pilihan model yang kuat.

Practical note. SVM sangat sensitif terhadap skala fitur. Gunakan StandardScaler dalam Pipeline agar scaling hanya di-fit pada training data dan diterapkan konsisten saat cross-validation maupun testing.

2 Hyperplane, Margin, dan Support Vectors

2.1 Hyperplane

Hyperplane adalah decision boundary dalam ruang fitur. Pada data 2D bentuknya garis, pada data 3D bentuknya bidang, dan pada dimensi lebih tinggi disebut hyperplane. Formulasnya:

$$w^T x + b = 0 \quad (1)$$

Prediction rule untuk binary classification:

$$f(x) = \text{sign}(w^T x + b) \quad (2)$$

Jika nilai fungsi positif, data masuk class positif. Jika negatif, data masuk class negatif.

2.2 Margin dan Support Vectors

Margin adalah jarak aman antara hyperplane dan data terdekat dari setiap class. Untuk data linearly separable, lebar margin berhubungan dengan:

$$\text{margin} = \frac{2}{\|w\|} \quad (3)$$

Karena itu, maximize margin setara dengan minimize $\|w\|^2$. Data points yang berada paling dekat dengan hyperplane disebut *support vectors*. Mereka adalah data yang benar-benar menentukan posisi boundary.

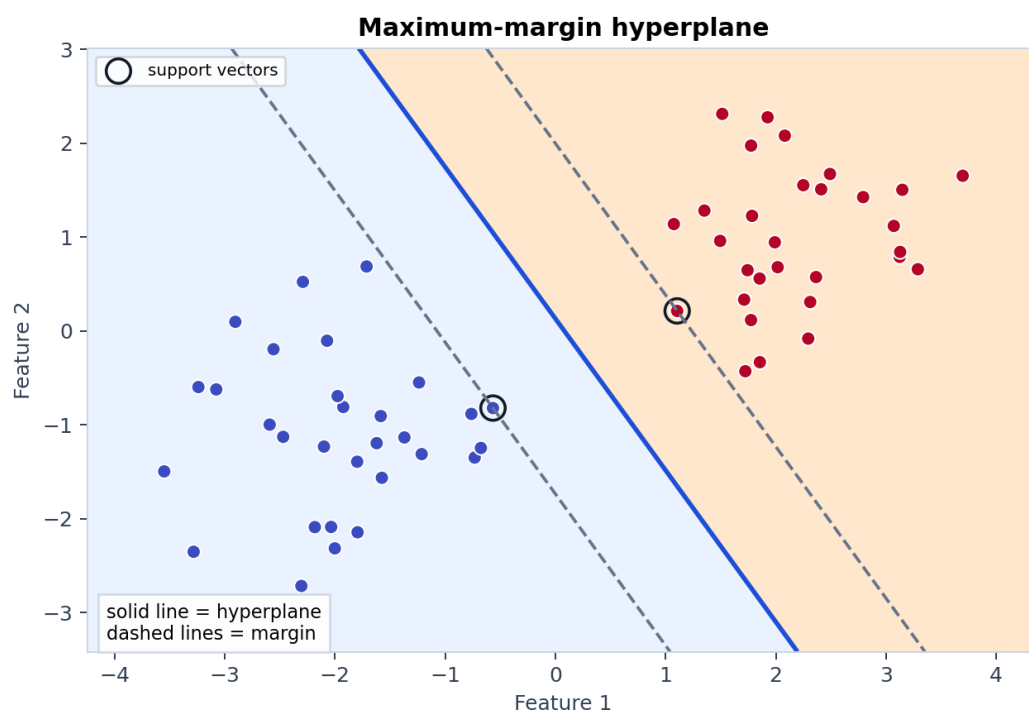


Figure 1: Maximum-margin hyperplane. Garis penuh adalah decision boundary, garis putus-putus adalah margin, dan lingkaran hitam menunjukkan support vectors.

Key insight. Tidak semua data training punya pengaruh yang sama. Pada SVM, boundary terutama ditentukan oleh support vectors, yaitu data paling dekat dengan margin.

3 Hard Margin dan Soft Margin

3.1 Hard Margin

Hard margin mengasumsikan data bisa dipisahkan sempurna tanpa error. Objective-nya:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad (4)$$

Dengan constraint:

$$y_i(w^T x_i + b) \geq 1 \quad (5)$$

Hard margin jarang cocok untuk real-world data karena data sering punya noise, overlap, atau outlier.

3.2 Soft Margin

Soft margin memperbolehkan sebagian data masuk ke margin atau bahkan salah klasifikasi. Pelanggaran ini diukur dengan slack variable ξ_i dan dikontrol oleh parameter C.

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (6)$$

Subject to:

$$y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (7)$$

Nilai C	Efek	Risiko
Kecil	Margin lebih lebar dan lebih toleran terhadap error.	Bisa underfit.
Besar	Error dipenalti lebih keras dan boundary mengejar training data.	Bisa overfit.

Table 2: Interpretasi parameter C pada soft-margin SVM.

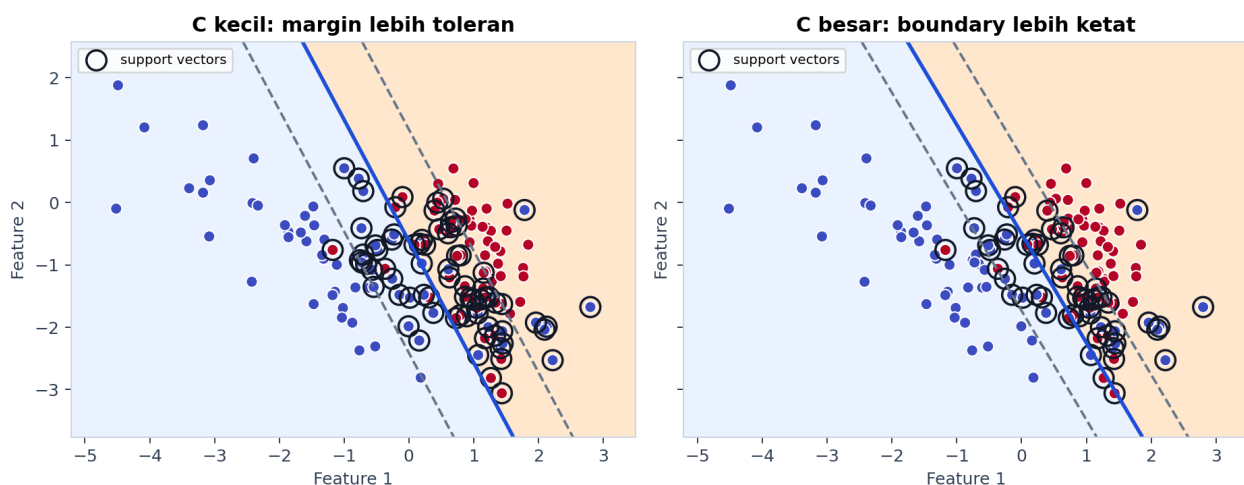


Figure 2: Efek parameter C. Nilai C kecil membuat boundary lebih toleran, sedangkan C besar membuat boundary lebih ketat terhadap training data.

4 Convex Optimization Intuition

SVM training menyelesaikan optimization problem yang convex. Artinya, objective SVM punya global optimum yang bisa dicari secara stabil. Secara praktis, SVM menggabungkan objective margin dan penalty error.

Komponen Objective	Makna
$\frac{1}{2}\ w\ ^2$	Membuat margin besar dan model lebih sederhana.
$C \sum \xi_i$	Penalize data yang melanggar margin atau salah klasifikasi.

Soft-margin linear SVM juga dapat dilihat melalui hinge loss:

$$\ell_i = \max(0, 1 - y_i(w^T x_i + b)) \quad (8)$$

Jika point benar dan jauh di luar margin, loss bernilai 0. Jika point berada di dalam margin atau salah klasifikasi, loss menjadi positif.

Teaching intuition. SVM bukan model yang sekadar “mencari garis”. Ia menyelesaikan masalah optimasi: cari boundary paling sederhana dengan margin besar, tetapi tetap mengontrol training mistake.

5 Support Vector Classification

5.1 SVC di scikit-learn

SVC adalah implementasi Support Vector Classification berbasis LIBSVM. Ia mendukung binary classification, multiclass classification, kernel linear maupun non-linear, dan class weighting untuk imbalanced data.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

5.2 SVC vs LinearSVC

Aspek	SVC	LinearSVC
Backend	LIBSVM	LIBLINEAR
Kernel	linear, poly, rbf, sigmoid, custom	linear only
Cocok untuk	Dataset kecil-menengah dan boundary non-linear	Dataset besar atau high-dimensional sparse data
Support vectors	Memiliki support_vectors_	Tidak sama seperti SVC

Table 3: Perbandingan ringkas SVC dan LinearSVC.

6 Kernel Trick dan Kernel Functions

Kernel trick memungkinkan SVM menghitung inner product di feature space tanpa membuat mapping fitur secara eksplisit.

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad (9)$$

Decision function SVM berbasis kernel bergantung pada support vectors:

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b \quad (10)$$

Kernel	Formula	Kapan Dipakai
Linear	$K(x, z) = x^T z$	Data cukup linear atau high-dimensional sparse.
Polynomial	$K(x, z) = (\gamma x^T z + r)^d$	Ingin menangkap interaksi fitur sampai degree tertentu.
RBF	$K(x, z) = \exp(-\gamma \ x - z\ ^2)$	Default kuat untuk boundary non-linear.
Sigmoid	$K(x, z) = \tanh(\gamma x^T z + r)$	Jarang menjadi pilihan pertama; sensitif parameter.

Table 4: Kernel function umum pada SVC.

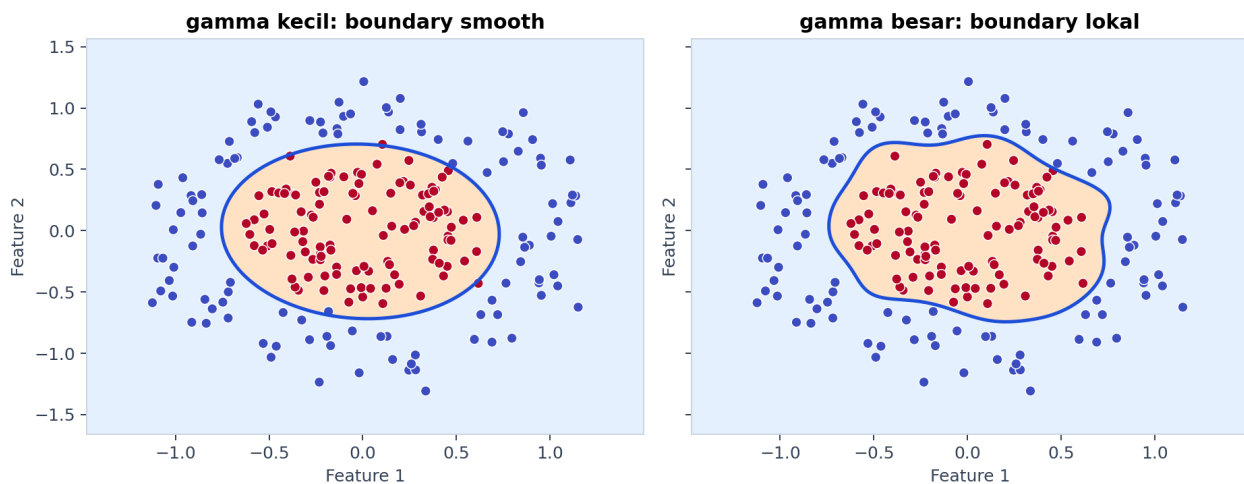


Figure 3: Efek parameter gamma pada RBF kernel. Gamma kecil menghasilkan boundary smooth, sedangkan gamma besar membuat boundary sangat lokal dan mudah overfit.

7 Parameter Penting SVC

Parameter	Berlaku Untuk	Makna
C	semua kernel	Penalty untuk training error dan margin violation.
kernel	semua	Jenis similarity function yang dipakai.
gamma	rbf, poly, sigmoid	Seberapa jauh pengaruh satu training point.
degree	poly	Derajat polynomial kernel.
coef0	poly, sigmoid	Independent term dalam kernel.
class_weight	classification	Bobot class, berguna untuk imbalanced data.
probability	SVC	Mengaktifkan probability estimate, tetapi training menjadi lebih mahal.

Table 5: Parameter penting dalam `sklearn.svm.SVC`.

Rule of thumb. Mulai dari Pipeline(StandardScaler(), SVC(kernel="rbf")), lalu tune C dan gamma. Jika dataset sangat besar atau text-like, pertimbangkan LinearSVC atau SGDClassifier.

8 Mencari Parameter Optimal

8.1 Grid Search

Grid Search mencoba semua kombinasi parameter dalam grid. Untuk SVC, grid yang umum adalah kombinasi kernel, C, dan gamma. Pada pipeline, parameter ditulis dengan pola step__parameter, misalnya svc__C.

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC())
])

param_grid = [
    {"svc__kernel": ["linear"], "svc__C": [0.1, 1, 10, 100]},
    {"svc__kernel": ["rbf"], "svc__C": [0.1, 1, 10, 100],
     "svc__gamma": [0.001, 0.01, 0.1, 1]},
]

search = GridSearchCV(pipe, param_grid, cv=5,
                      scoring="accuracy", n_jobs=-1)
search.fit(X_train, y_train)
```

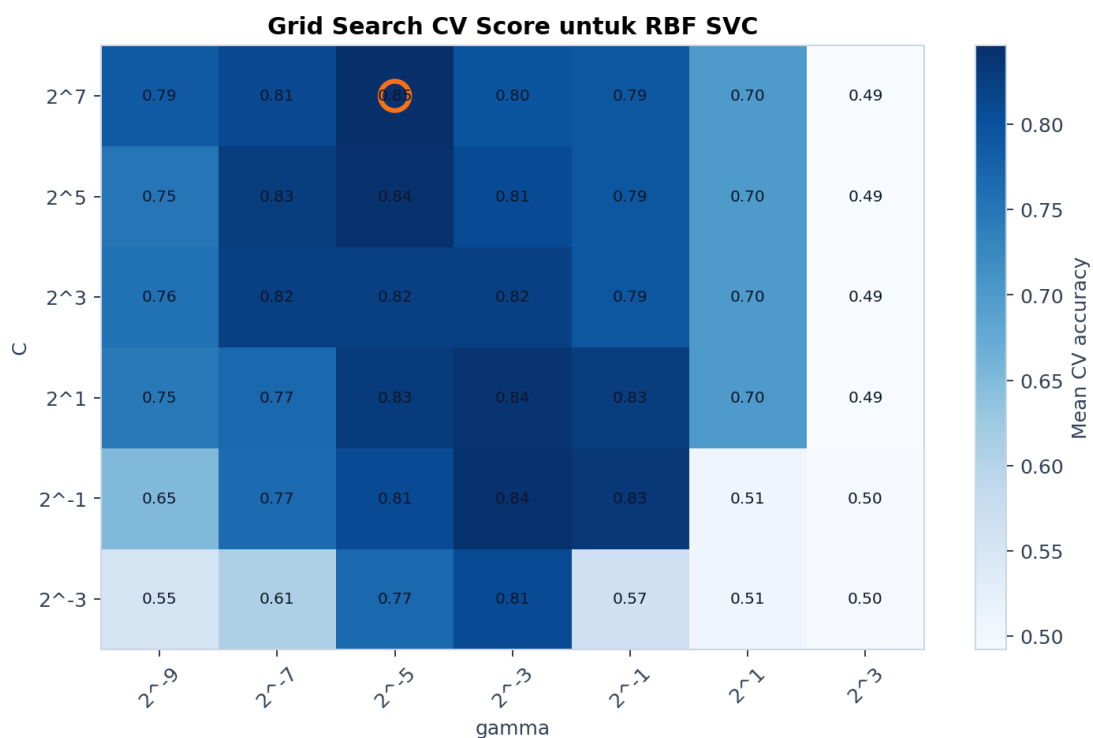


Figure 4: Contoh heatmap Grid Search untuk RBF SVC. Nilai C dan gamma dicoba pada skala log, lalu kombinasi terbaik dipilih berdasarkan cross-validation score.

8.2 Random Search

Random Search sampling kombinasi parameter dari distribusi. Ini cocok ketika search space besar dan kita ingin mengontrol budget komputasi dengan `n_iter`.

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "svc__kernel": ["rbf"],
    "svc__C": loguniform(1e-3, 1e3),
    "svc__gamma": loguniform(1e-4, 1e1),
}

search = RandomizedSearchCV(
    pipe, param_distributions=param_dist,
    n_iter=50, cv=5, scoring="accuracy",
    n_jobs=-1, random_state=42
)
```

8.3 Optuna

Optuna mendefinisikan search space secara Pythonic. Cocok ketika parameter bersifat conditional, misalnya `gamma` hanya dicoba untuk kernel tertentu.

```
import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    C = trial.suggest_float("C", 1e-3, 1e3, log=True)
    gamma = trial.suggest_float("gamma", 1e-4, 1e1, log=True)

    model = Pipeline([
        ("scaler", StandardScaler()),
        ("svc", SVC(kernel="rbf", C=C, gamma=gamma))
    ])

    scores = cross_val_score(model, X_train, y_train,
                             cv=5, scoring="accuracy")
    return scores.mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)
```

Metode	Kelebihan	Kekurangan
Grid Search	Exhaustive dan mudah dijelaskan.	Mahal jika kombinasi banyak.
Random Search	Budget fleksibel dan cocok untuk parameter continuous.	Bisa miss region bagus jika trial sedikit.
Optuna	Fleksibel, conditional, dan trial-based.	Perlu library tambahan dan konsep lebih advanced.

Table 6: Perbandingan metode hyperparameter tuning untuk SVC.

9 Practical Workflow dan Common Mistakes

Workflow yang direkomendasikan:

1. Split data menjadi training dan testing set.
2. Buat pipeline `StandardScaler + SVC`.

3. Tentukan metric sesuai problem, misalnya accuracy, F1, recall, ROC-AUC, atau balanced accuracy.
4. Jalankan baseline model.
5. Tune parameter dengan Grid Search, Random Search, atau Optuna pada training set.
6. Refit model terbaik pada training set.
7. Evaluasi final pada test set.

Mistake	Dampak	Fix
Tidak scaling fitur	Feature range besar mendominasi kernel.	Pakai StandardScaler dalam Pipeline.
Tuning di test set	Evaluasi final terlalu optimis.	Tune dengan cross-validation pada train set.
Grid terlalu sempit	Parameter terbaik tidak ditemukan.	Gunakan log-spaced range luas dulu.
C terlalu besar	Boundary overfit.	Coba C lebih kecil.
gamma terlalu besar	Boundary terlalu lokal.	Coba gamma lebih kecil.
Accuracy untuk imbalance	Metric misleading.	Pakai F1, recall, ROC-AUC, atau balanced accuracy.

Table 7: Kesalahan umum saat menggunakan SVC dan cara menghindarinya.

10 Latihan

1. Diberikan dua decision boundary yang sama-sama memisahkan class. Jelaskan boundary mana yang lebih baik menurut SVM dan kenapa.
2. Jelaskan efek kombinasi C kecil-besar dan gamma kecil-besar terhadap boundary RBF SVC.
3. Buat pipeline StandardScaler → SVC, lalu lakukan GridSearchCV untuk kernel linear dan RBF.
4. Jika dataset memiliki class ratio 95:5, kenapa accuracy tidak cukup? Metric apa yang lebih cocok?

Referensi

- Cortes, C. & Vapnik, V. (1995). *Support-Vector Networks*. Machine Learning, 20, 273–297.
- Boser, B. E., Guyon, I. M. & Vapnik, V. N. (1992). *A Training Algorithm for Optimal Margin Classifiers*.
- Hsu, C.-W., Chang, C.-C. & Lin, C.-J. (2025). *A Practical Guide to Support Vector Classification*.
- Chang, C.-C. & Lin, C.-J. (2011). *LIBSVM: A Library for Support Vector Machines*.
- Bergstra, J. & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*.
- scikit-learn SVM User Guide: <https://scikit-learn.org/stable/modules/svm.html>
- scikit-learn hyperparameter tuning: https://scikit-learn.org/stable/modules/grid_search.html
- Optuna documentation: <https://optuna.readthedocs.io/>